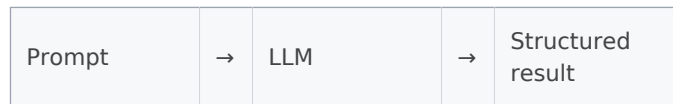


LangChain — Lecture 6

Beginner-Friendly Guide to Advanced Prompt Engineering & Output Parsers

What you will learn

Chain-of-Thought prompting • Partial Prompt Templates • Message Placeholders • Structured Output • Pydantic Output Parser

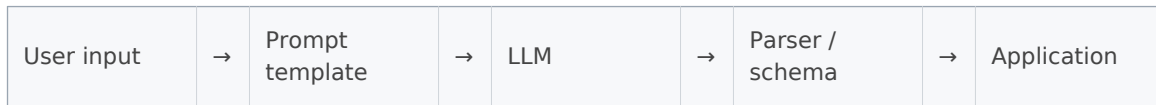


Goal: Understand not only what these LangChain features do, but also why they are useful and when you would use them in a real application.

Based on: Lecture 6 of the LangChain Developer Course and the supplied video summary.

1. Big Picture

This lecture moves from basic prompting toward more controlled LLM applications. The central idea is simple: instead of sending a loose question to an LLM and hoping for a useful answer, we can design the prompt and expected output more carefully.



The lecture introduces three prompt-engineering techniques and two ways to get structured output.

Topic	Main purpose
Chain-of-Thought prompting	Encourage step-by-step reasoning for problems that benefit from explicit reasoning.
Partial prompt templates	Pre-fill known variables while keeping other variables dynamic.
Message placeholders	Insert dynamic content such as chat history into a prompt.
Structured output	Ask the model to return data following a defined schema.
Pydantic output parser	Define and validate structured data with a Pydantic model.

Key takeaway: Good LLM applications control both the input instructions and the shape of the output.

2. Chain-of-Thought (CoT) Prompting

Chain-of-Thought prompting is a technique where a prompt encourages an LLM to work through a problem in a step-by-step manner rather than immediately producing a final answer.

Why use it? Some tasks involve several decisions or calculations. Asking for a structured reasoning process can make the model's approach more organized and can improve performance on suitable tasks.

Example: insurance claim

Imagine an application that receives an insurance claim. It may need to examine the incident, policy details, exclusions, evidence, and finally decide whether the claim appears eligible.



Simple prompt idea

You are an insurance claim assistant.

Analyze the claim step by step.

Consider:

1. What happened?
2. What does the policy cover?
3. Are there exclusions?
4. What evidence is available?
5. What is the final conclusion?

Important beginner point: CoT is about prompting the model to reason through a problem. In production systems, you generally should not assume that exposing private/internal reasoning is necessary. A better application pattern is often to request a concise explanation, decision, or structured justification instead.

When it helps: multi-step reasoning, classification with several criteria, planning, calculations, troubleshooting.

When it may be unnecessary: simple factual questions where a direct answer is enough.

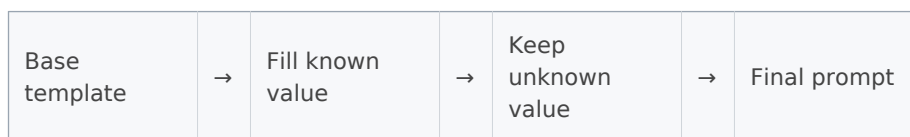
Conceptual comparison

Direct prompt	Reasoning-oriented prompt
"Is this claim valid?"	"Evaluate the claim using these criteria and provide the conclusion with a concise justification."
Short and simple	More guidance for complex decisions
Remember: The goal is not to make every prompt longer. Use extra reasoning guidance when the task actually benefits from it.	

3. Partial Prompt Templates

A prompt template can contain variables. A partial prompt template lets you fill in some variables now and leave the remaining variables to be supplied later.

Think of it like a form where some fields are already completed.



Example

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "You are an expert in {domain}. Answer this question: {question}"
)

partial_prompt = prompt.partial(domain="insurance")

# Later:
final_prompt = partial_prompt.invoke({
    "question": "What is a deductible?"
})
```

Here, domain is known ahead of time, so it can be pre-filled. question remains dynamic because it changes with each user request.

Variable	Example value	When supplied?
domain	insurance	Before user request / application setup
question	What is a deductible?	At runtime

Why is this useful?

Partial templates are helpful when your application has a fixed role, domain, language, company policy, formatting rule, or other configuration that should not be repeated manually every time.

Easy way to remember: Partial template = “Fill what I already know; ask for the rest later.”

4. Message Placeholders

Real chat applications have changing context. A user may have sent several messages before asking the current question. A message placeholder gives the prompt a place where dynamic messages can be inserted.

Why not hardcode chat history? Because the history changes from conversation to conversation. The application needs a reusable prompt that can receive the current history at runtime.



Conceptual example

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder("history"),
    ("human", "{question}")
])
```

At runtime, history can contain previous human and AI messages. The same prompt template can therefore work across many conversations.

Prompt part	Typical content
System message	Rules, role, behavior
MessagesPlaceholder	Dynamic conversation history
Human message	Current user question
Key takeaway: A placeholder is a slot for information that your application will provide dynamically later.	

5. Structured Output

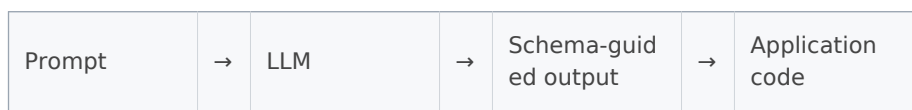
Normally, an LLM may answer in natural language. That is useful for people, but software often needs predictable fields such as name, age, category, or score. Structured output addresses this problem.

Unstructured response

The customer appears eligible. Their age is 31 and the policy type is health insurance.

Structured response

```
{
  "customer_name": "Rahul",
  "age": 31,
  "policy_type": "health"
}
```



with_structured_output

LangChain chat models can provide a `with_structured_output` interface that tells the model what structure the application expects. The exact supported behavior depends on the model/provider.

```
class UserInfo(BaseModel):
    name: str
    age: int
    city: str

structured_llm = llm.with_structured_output(UserInfo)

result = structured_llm.invoke(
    "My name is Amit, I am 28 and live in Pune."
)
```

The important idea is that the application defines the desired structure first, then asks the model to produce a result matching it.

Why structured output matters: predictable data is much easier for downstream code to process than free-form text.

6. Pydantic Output Parser

Pydantic is a Python library commonly used to define data models and validate data using Python type hints. In LangChain, a Pydantic model can describe what the LLM output should look like.

Example model

```
from pydantic import BaseModel, Field

class ProductReview(BaseModel):
    product: str
    rating: int = Field(description="Rating from 1 to 5")
    sentiment: str
```

summary: str

This model tells us that a product review should contain four fields. The rating should be an integer, while the other fields are strings.

Using the model with an output parser

```
from langchain_core.output_parsers import PydanticOutputParser

parser = PydanticOutputParser(
    pydantic_object=ProductReview
)

format_instructions = parser.get_format_instructions()

prompt = PromptTemplate.from_template(
    "Analyze this review: {review}\n{format_instructions}"
)

final_prompt = prompt.invoke({
    "review": "The headphones sound great, but the battery is weak.",
    "format_instructions": format_instructions
})

response = llm.invoke(final_prompt)
result = parser.parse(response.content)
```

What happens?



Part	Job
Pydantic model	Defines expected fields and types
Format instructions	Communicates the desired output format to the model
LLM	Generates the response
Pydantic parser	Converts/validates the model response

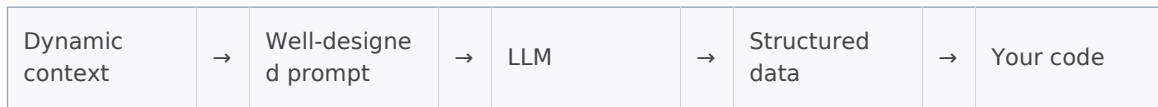
Beginner distinction: with `structured_output` is a convenient model-level approach for structured results. `PydanticOutputParser` is a parser-based approach where a Pydantic schema defines the expected result and the parser handles the model output.

7. Putting the Concepts Together

These topics are easier to understand when viewed as pieces of one application.

Requirement	Useful feature	Why
Complex multi-step task	Reasoning-oriented prompting	Provides more guidance for the task
Some prompt values are fixed	Partial template	Pre-fills known information
Conversation history changes	Message placeholder	Inserts dynamic messages

Software needs predictable data	Structured output	Produces a defined data shape
Need typed/validated Python data	Pydantic	Defines fields and expected types



8. Quick Revision / Cheat Sheet

Term	Remember it as...
Chain-of-Thought prompting	Guide the model through a complex task step-by-step; prefer concise justifications in production when appropriate.
Partial prompt	Pre-fill some variables now; supply the rest later.
Message placeholder	A slot for dynamic messages such as chat history.
Structured output	Make the LLM response follow a defined data shape.
Pydantic	Define Python data models with fields and types; useful for validation and structured LLM results.
with_structured_output	Ask a supported chat model for a schema-guided structured result.
PydanticOutputParser	Parse an LLM response according to a Pydantic model.

9. Beginner Practice Exercise

Build a tiny “Product Review Analyzer” application. Give it a review and ask it to return a structured result.

Input:

"The phone looks good and is fast, but the battery is disappointing."

Expected structure:

```
{
  "product": "...",
  "rating": 1-5,
  "sentiment": "...",
  "summary": "..."
}
```

Try these improvements one at a time:

1. Create a Pydantic model for the output.
2. Create a prompt template for the review.
3. Add format instructions or use structured output.
4. Invoke the LLM.
5. Inspect the resulting Python object.
6. Change the review and test again.

Learning strategy: Don't memorize every class name. First understand the flow: input → prompt → model → structured result. Then learn the LangChain syntax used to implement each step.

10. What Comes Next?

According to the supplied lecture summary, additional output parsers such as JSON, CSV, and Output Fixing are covered in Part 2.

Final takeaway: Lecture 6 is mainly about making LLM applications more reliable and reusable: better prompts for complex tasks, reusable templates for known configuration, dynamic placeholders for conversation context, and structured outputs that software can actually work with.